Git e GitHub CLI na prática

Do primeiro commit à publicação do projeto



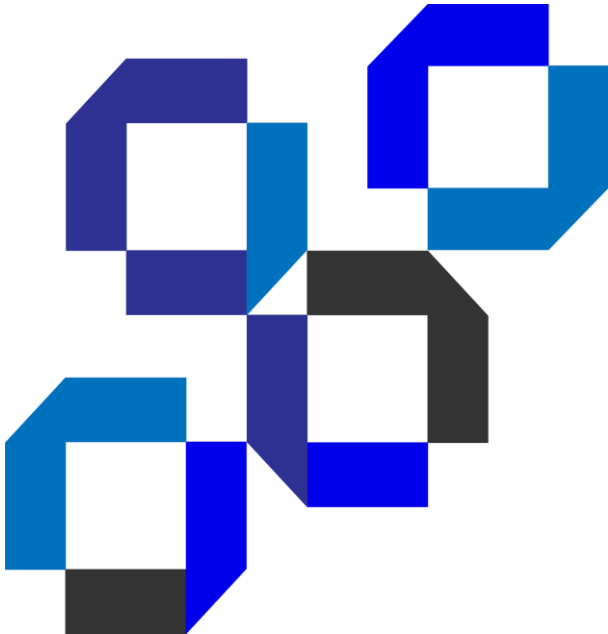


Guilherme Martins, Diogo Simioli, Victor Afonso,
Tiago Arthur e Miguel Ranieri

TechnoMonte 2026 – Monte Carmelo - MG

Git e GitHub CLI na prática

- Abertura e verificação do ambiente
- Fundamentos
- Demonstração dos comandos básicos
- Branches e integração
- Recuperação de uma alteração
- Prática de resolução de conflito
- Prática de recuperação e publicação
- Verificação e dúvidas



Confira o ambiente antes de começar

Instalação e autenticação são pré-requisitos do workshop.

Saída correta: duas versões, seu nome, seu e-mail e "Logged in".





```
$ git --version; gh --version
git version 2.43.0
gh version 2.45.0
$ git config user.name; git config user.email
Maria Silva
maria@exemplo.com
$ gh auth status
✓ Logged in to github.com account maria-silva
```

Git, GitHub e GitHub CLI: três peças diferentes



Git

Ferramenta local que guarda as versões do projeto.

Git é o caderno



GitHub

Plataforma online que guarda e compartilha repositórios.

GitHub, a estante



GitHub CLI

Ferramenta para usar o GitHub pelo terminal.

CLI, a ponte pelo terminal

Por que controlar versões?

Problema: arquivo_final_v2, final_agora_vai...

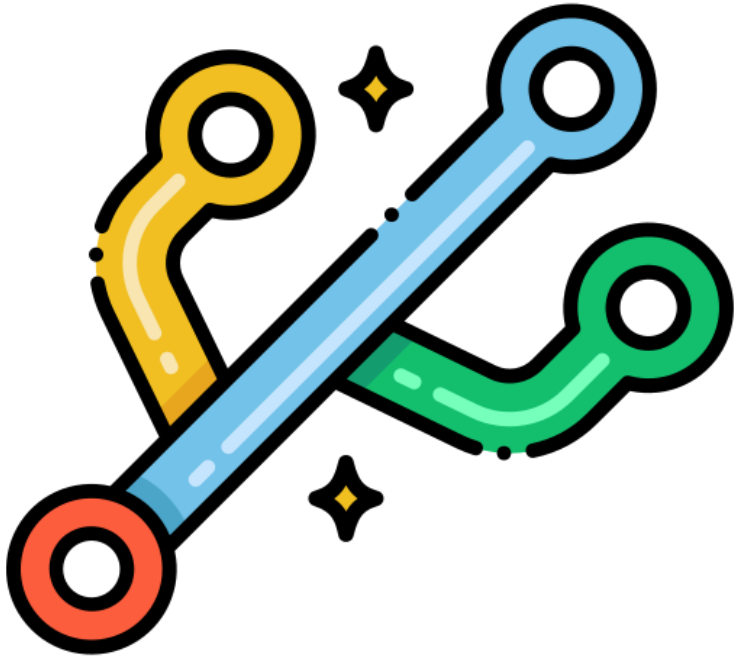
Com Git: um só arquivo e um histórico.

Problema: uma alteração foi perdida.

Com Git: volte a qualquer versão registrada.

Problema: ninguém sabe quem mudou o quê.

Com Git: cada versão guarda autor, data e mensagem.

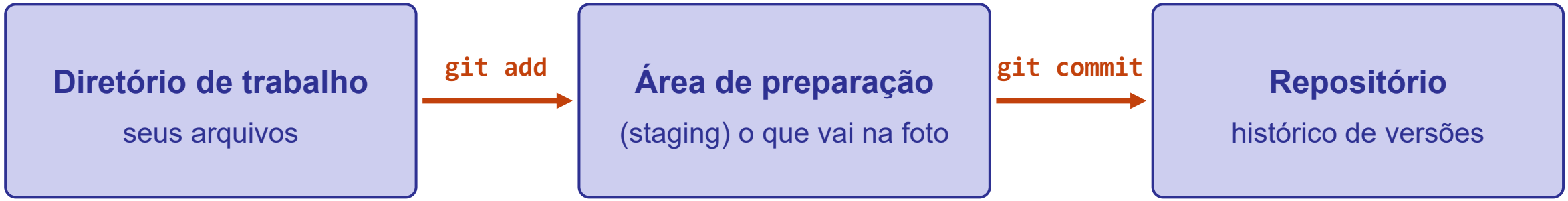


Três áreas: do arquivo editado ao histórico

O fluxo local de arquivos é dividido em três estados claros:

- Diretório de Trabalho (Working Directory): arquivos em edição.
- Área de Preparação (Staging Area): caixa de arquivos para o commit.
- Repositório (.git): histórico definitivo com snapshots (fotos) salvas.



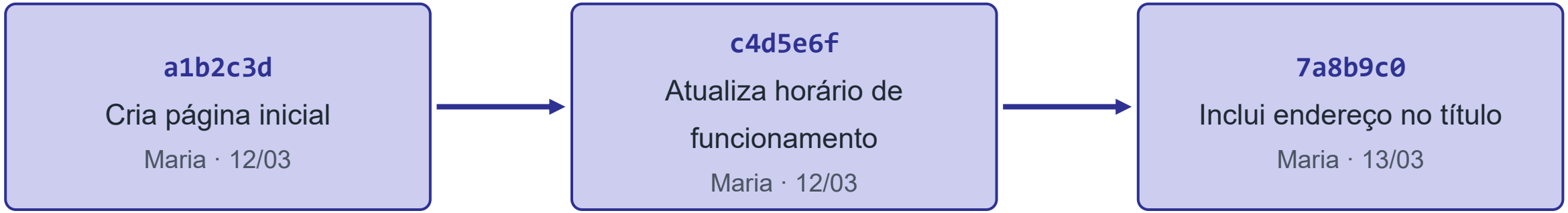


Um commit é uma foto datada do projeto

Commit (registro): snapshot (foto) datada do projeto.

Guarda autor, data, mensagem e identificador.

Boa mensagem: curta, no imperativo, diz o que mudou.



o tempo avança da esquerda para a direita

git init e git status: crie e confira

“git init” cria o repositório (use uma vez por projeto).

“git status” mostra o estado dos arquivos (use sempre).

untracked: não rastreado.

staged: preparado.

committed: registrado.



```
$ git init
Initialized empty Git repository in /meu-projeto/.git/
$ git status
On branch main
Untracked files:
        index.html
```

git add e git commit: registre a versão

“git add” coloca o arquivo na área de preparação (staging).

“git commit –m” grava a versão com uma mensagem.



```
$ git add index.html
$ git commit -m "Cria estrutura inicial de index.html"
[main (root-commit) 7a1b2c3] Cria estrutura inicial de index.html
 1 file changed, 10 insertions(+)
$ git status
nothing to commit, working tree clean
```

git log --oneline: leia o histórico

git log --oneline lista as versões em uma linha cada.

1ª coluna: identificador curto do commit.

2ª coluna: a mensagem de quem registrou.

O commit mais recente aparece primeiro.



```
$ git log --oneline
$ git log --oneline
7a1b2c3 (HEAD -> main) Cria estrutura inicial de index.html
3f8e9d2 Adiciona seção de contato no rodape
1a2b3c4 Configura cabeçalho principal do site
```

git diff: veja o que mudou antes de registrar

“git diff” compara o arquivo com a última versão registrada.

- linha removida (vermelho).

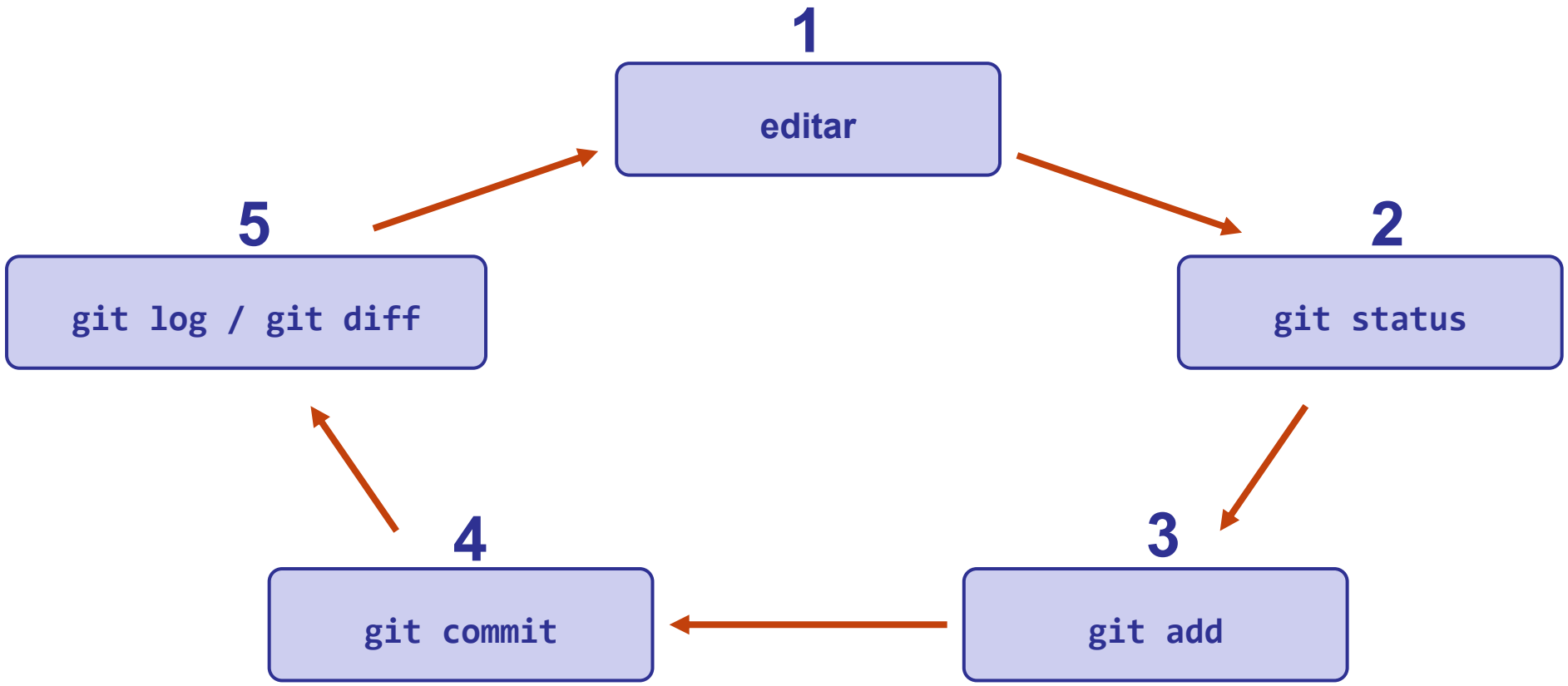
+ linha adicionada (verde).

Use antes do add, para revisar o que você mudou.



```
$ git diff
--- a/index.html
+++ b/index.html
@@ -5,3 +5,3 @@
-<h1>Título Antigo</h1>
+<h1>Workshop Git e GitHub CLI</h1>
```

O ciclo do dia a dia



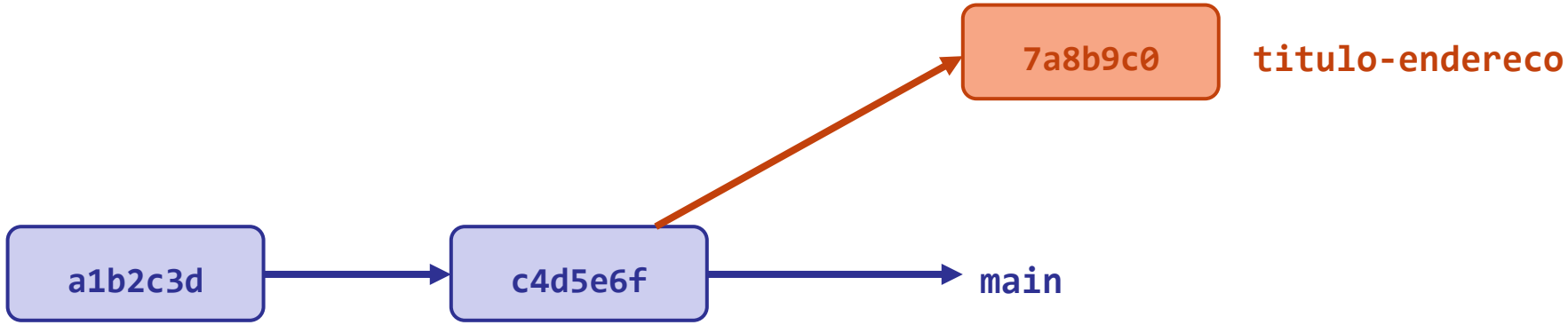
Branch: uma cópia de trabalho paralela

Branch (ramo): cópia de trabalho paralela.

A main é a linha principal;

o ramo sai de um commit.

Teste ideias sem mexer na main.



git branch e git switch: crie e alterne

“git branch nome_do_ramo” cria o ramo.

“git switch nome_do_ramo” muda para ele.

“git branch”, sem nome, lista os ramos. O * marca o atual.

Use um ramo para cada ideia ou correção.



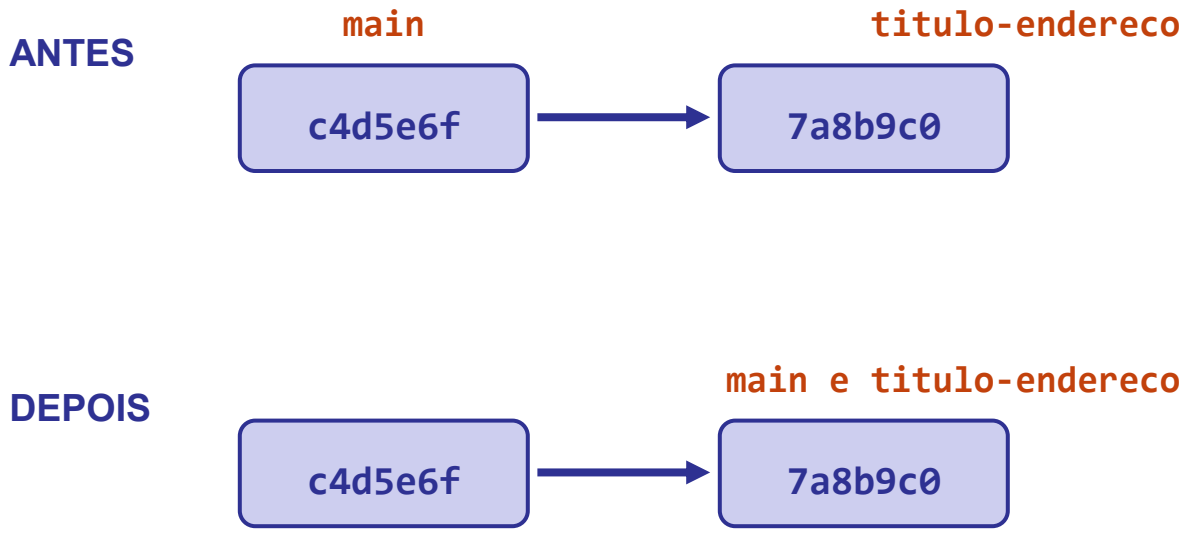
```
$ git branch titulo-endereco
$ git switch titulo-endereco
Switched to branch 'titulo-endereco'
$ git branch
  main
* titulo-endereco
```

git merge: traga o ramo para a main

“git merge nome_do_ramo” integra (merge) o ramo na branch atual.

Rode a partir da branch de destino: aqui, a main.

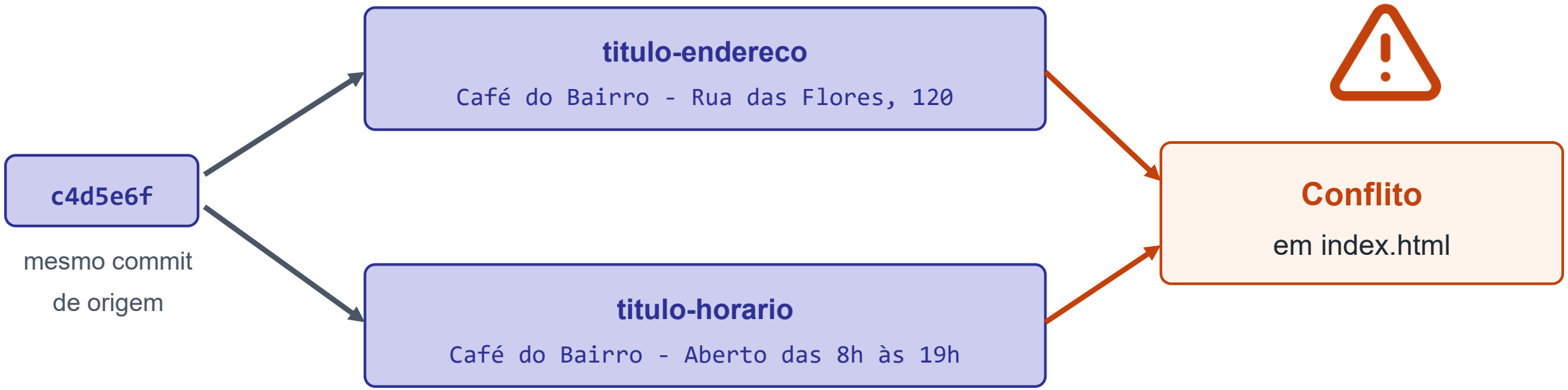
```
$ git switch main
$ git merge titulo-endereco
Updating c4d5e6f..7a8b9c0
Fast-forward
 index.html | 2 +-
```



Conflito: dois ramos mudaram a mesma linha

Conflito: dois ramos alteraram a mesma linha do <h1>.

O Git não decide: ele pede que você escolha.



Os marcadores mostram as duas versões

O Git marcou as duas versões dentro de index.html:

```
<<<<<<< HEAD
<h1>Café do Bairro - Rua das Flores, 120</h1>
=======
<h1>Café do Bairro - Aberto das 8h às 19h</h1>
>>>>>>> titulo-horario
```



`<<<<<<< HEAD`	`=======`	`>>>>>>> nome_do_ramo`
versão atual (main)	separador	versão que chega

Resolva em quatro passos curtos

1. Abra index.html e decida o texto final do título.
2. Apague as linhas <<<<<<<, ======= e >>>>>>>.
3. Salve e prepare: git add index.html
4. Registre: git commit -m "Resolve conflito no título"



O arquivo final não pode conter marcadores.

Exemplo de título final em index.html:

```
<h1>Café do Bairro - Rua das Flores, 120 - 8h às 19h</h1>
```

git revert desfaz sem apagar o histórico

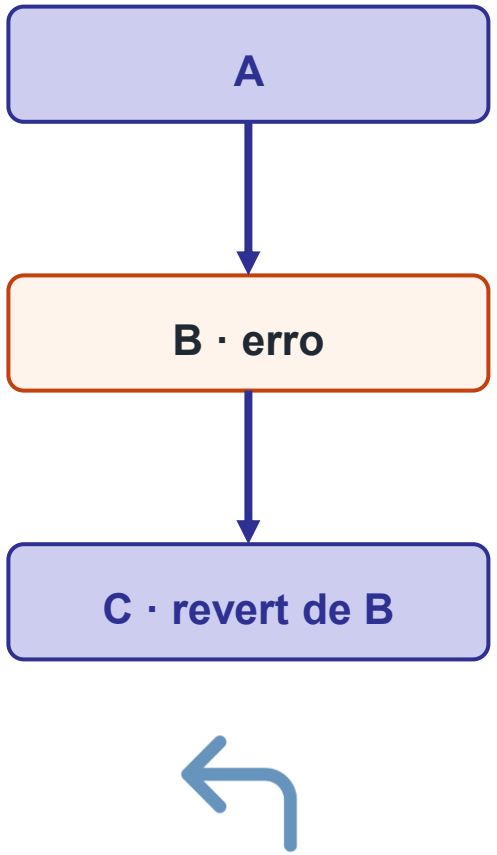
Problema: um commit trouxe um erro.

Solução: um novo commit o desfaz.

Vantagem: histórico completo.

Analogia: nova foto, álbum intacto.

```
$ git log --oneline
e5f6a7b Marca café como fechado
4b5c6d7 Resolve conflito no título
$ git revert e5f6a7b
[main 3f2e1d0] Revert "Marca café como fechado"
$ git log --oneline
3f2e1d0 Revert "Marca café como fechado"
e5f6a7b Marca café como fechado
```



gh auth: conecte o terminal ao GitHub

“gh auth login” conecta o terminal à sua conta do GitHub.

“gh auth status” confirma se a conexão está ativa.

Confira na saída: "Logged in" e o nome da sua conta.



```
$ gh auth login
? What account do you want to log into? GitHub.com
? How would you like to authenticate? Login with a web browser
✓ Authentication complete.
$ gh auth status
✓ Logged in to github.com account maria-silva (keyring)
```

gh repo create: publique pelo terminal

--source=. usa a pasta atual como origem.

--remote=origin nomeia o endereço remoto como origin.

--push envia os commits já registrados.

--public ou --private: quem pode ver.

Rode dentro da pasta do projeto, com tudo registrado.





```
$ gh repo create pagina-cafe --source=. --remote=origin --push --public
✓ Created repository maria-silva/pagina-cafe on GitHub
✓ Added remote https://github.com/maria-silva/pagina-cafe.git
✓ Pushed commits to https://github.com/maria-silva/pagina-cafe.git
```

gh repo view --web: confira no navegador

gh repo view --web abre o repositório no navegador.

Confira: arquivos e histórico de commits, com mensagens.

Exemplo de outros comandos do gh: gh pr create.



```
$ gh repo view --web
Opening github.com/maria-silva/pagina-cafe in your browser.
```

gh pr create –title "título": cria pull requests

O comando ghpr create –title "" cria um novo Pull Request (PR) no GitHub diretamente pelo terminal com o título escrito em "".



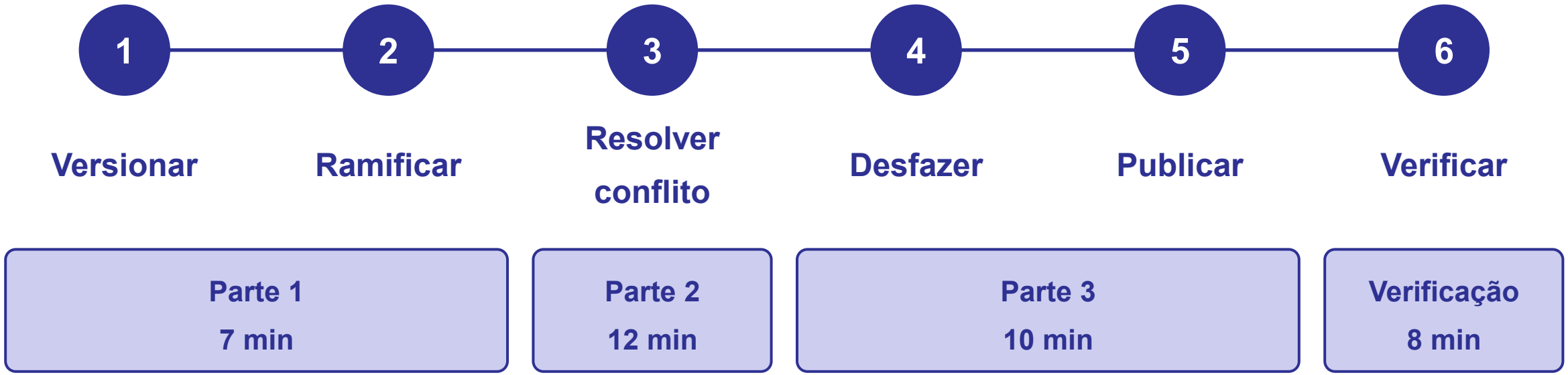
Por padrão, ele envia as alterações da sua branch atual para o repositório remoto e abre um menu interativo no terminal para você definir o título e a descrição do PR.

```
$ gh pr create –title "nova pr“
Creating pull request for sua-branch into main in seu-usuario/seu-repositorio.
https://github.com
```

Atividade: resgate e publique uma versão

Resgate e publique uma versão do projeto.

Individual, com um index.html simples fornecido.



Parte 1: ramos paralelos com a mesma linha

1. Crie o repositório local e registre a versão inicial.
2. Crie o ramo titulo-endereco a partir da main.
3. Volte à main e crie o ramo titulo-horario.
4. Em cada ramo, altere a mesma linha do <h1>.
5. Faça um commit por ramo.



Parte 2: integre, ache o conflito e resolva

1. Vá para a main: git switch main.
2. git merge titulo-endereco avança sem conflito.
3. git merge titulo-horario avisa do conflito.
4. Abra index.html e analise as duas versões.
5. Escreva um título final com endereço e horário.
6. Remova os marcadores; git add e git commit.



Parte 3: erre, reverta e publique

1. Altere outro trecho do HTML de propósito e registre.
2. git log --oneline: localize o commit errado.
3. git revert identificador: desfaça esse commit.
4. gh repo create ... --push: publique o projeto.
5. gh repo view --web: confira no navegador.





git commit –m “OBRIGADO PELA ATENÇÃO!”

facom
< Faculdade de Computação - UFU

UFU

technomonte 2026